

Първоначално стартиране

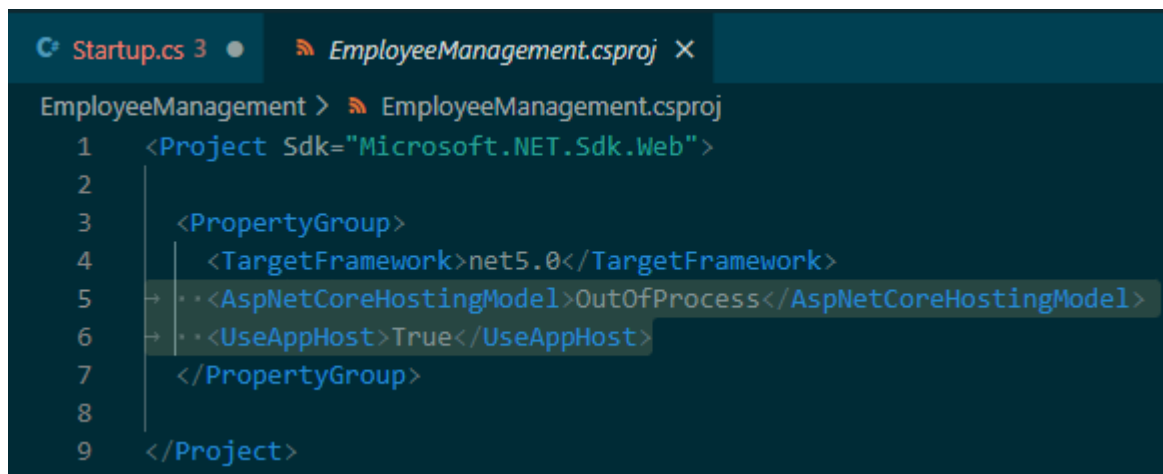
- 1) Във Visual Studio създаваме Empty проект без отметка за HTTPS версия 5.0 (съдържа Startup.cs с Main метод в него) и го наричаме „EmployeeManagement“. Затваряме VS.
- 2) Отваряме папката на проекта с Visual Code. За първоначално стартиране – отваряме терминала Open -> New Terminal -> навигираме до Program.cs файла, като евентуално може да се наложи първоначално да влезем във вътрешната директория с `cd .\` и Tab докато ни изпише папката Employee Management, потвърждаваме. Проектът се стартира с командата `dotnet run`. Когато изпише <http://localhost:5000> го зареждаме в браузъра. Всеки път, когато правим промяна в кода, натискаме
Ctrl + S = Save,
Ctrl + C = Clear (за да може да въведем нова команда).
`dotnet run` = стартиране на проекта
когато изпише <http://localhost:5000> го зареждаме в браузъра или най-добре той просто да си стои отворен в Mozilla и ние само да даваме Ctrl + R = refresh.

Като първа задача, в Startup.cs, на мястото на “HelloWorld” извикахме името на процеса в браузъра: `System.Diagnostics.Process.GetCurrentProcess().ProcessName`

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().ProcessName);
    });
});
```

И в EmployeeManagement.csproj дописахме 2 команди, с които тествахме процесите на локалните сървъри:

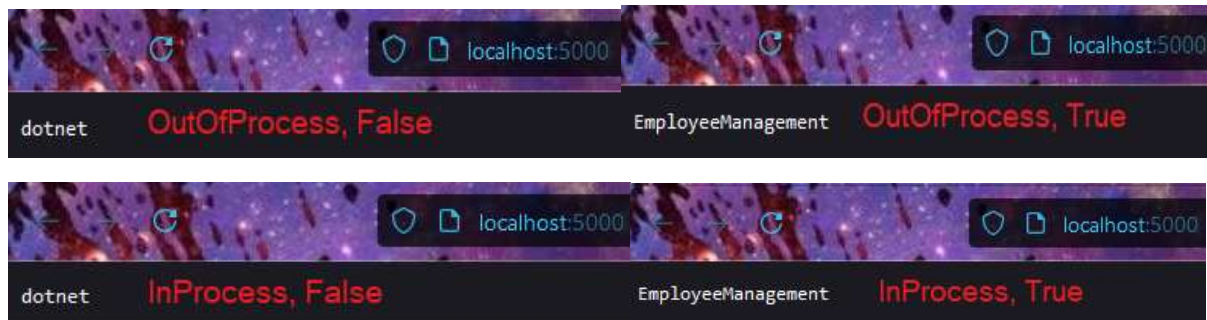
```
<AspNetCoreHostingModel>OutOfProcess</AspNetCoreHostingModel>
<UseAppHost>True</UseAppHost>
```



```
Startup.cs 3 • EmployeeManagement.csproj X
EmployeeManagement > EmployeeManagement.csproj
1 <Project Sdk="Microsoft.NET.Sdk.Web">
2
3 <PropertyGroup>
4 <TargetFramework>net5.0</TargetFramework>
5 <AspNetCoreHostingModel>OutOfProcess</AspNetCoreHostingModel>
6 <UseAppHost>True</UseAppHost>
7 </PropertyGroup>
8
9 </Project>
```

Като моделът за хостване може да има стойности `InProcess` или `OutOfProcess`, със съответно `True` или `False`. При стартиране в браузъра с <http://localhost:5000/> се изписва или името на проекта

EmployeeManagement, или dotnet, а ако дадем run във Visual Studio, имаме и бързодостъпен iis, който ще изпише iisexpress в брауъра.



Нека пробваме през CLI:

```
Developer Command Prompt for VS 2022
** Visual Studio 2022 Developer Command Prompt v17.10.4
** Copyright (c) 2022 Microsoft Corporation
*****
C:\Program Files\Microsoft Visual Studio\2022\Community>cd /d D:\Exercises\12_class\EmployeeManagement\EmployeeManagement
D:\Exercises\12_class\EmployeeManagement\EmployeeManagement>dotnet run
Using launch settings from D:\Exercises\12_class\EmployeeManagement\EmployeeManagement\Properties\launchSettings.json...
Building...

at Microsoft.AspNetCore.Server.Kestrel.Transport.Sockets.SocketTransportFactory.BindAsync(EndPoint endPoint, CancellationTok
onToken cancellationToken)
at Microsoft.AspNetCore.Server.Kestrel.Core.Internal.Infrastructure.TransportManager.BindAsync(EndPoint endPoint, ConnectionDe
legate connectionDelegate, EndpointConfig endpointConfig)
at Microsoft.AspNetCore.Server.Kestrel.Core.KestrelServerImpl.<>c__DisplayClass29_0`1.<<StartAsync>>g__OnBind|0>d.MoveNext()
--- End of stack trace from previous location ---
at Microsoft.AspNetCore.Server.Kestrel.Core.Internal.AddressBinder.BindEndpointAsync(ListenOptions endpoint, AddressBindin
dContext context)
--- End of inner exception stack trace ---
at Microsoft.AspNetCore.Server.Kestrel.Core.Internal.AddressBinder.BindEndpointAsync(ListenOptions endpoint, AddressBindin
dContext context)
at Microsoft.AspNetCore.Server.Kestrel.Core.LocalhostListenOptions.BindAsync(AddressBindContext context)
at Microsoft.AspNetCore.Server.Kestrel.Core.Internal.AddressBinder.AddressesStrategy.BindAsync(AddressBindContext conte
xt)
at Microsoft.AspNetCore.Server.Kestrel.Core.Internal.AddressBinder.BindAsync(IEnumerable`1 listenOptions, AddressBindCo
ntext context)
at Microsoft.AspNetCore.Server.Kestrel.Core.KestrelServerImpl.BindAsync(CancellationTokens cancellationTokens)
at Microsoft.AspNetCore.Server.Kestrel.Core.KestrelServerImpl.StartAsync[TContext](IHttpApplication`1 application, Canc
ellationToken cancellationToken)
at Microsoft.AspNetCore.Hosting.GenericWebHostService.StartAsync(CancellationTokens cancellationTokens)
at Microsoft.Extensions.Hosting.Internal.Host.StartAsync(CancellationTokens cancellationTokens)
at Microsoft.Extensions.Hosting.HostingAbstractionsHostExtensions.RunAsync(IHost host, CancellationTokens token)
at Microsoft.Extensions.Hosting.HostingAbstractionsHostExtensions.RunAsync(IHost host, CancellationTokens token)
at EmployeeManagement.Program.Main(String[] args) in D:\Exercises\12_class\EmployeeManagement\EmployeeManagement\Progra
m.cs:line 16
D:\Exercises\12_class\EmployeeManagement\EmployeeManagement>
```

!!! Много е важно да спрем стартирания процес от терминала с Ctrl + C не може 2 терминала едновременно да извикат един и същи процес. Ако единият във VS Code не е спрял, другият CLI ще хвърли грешка:

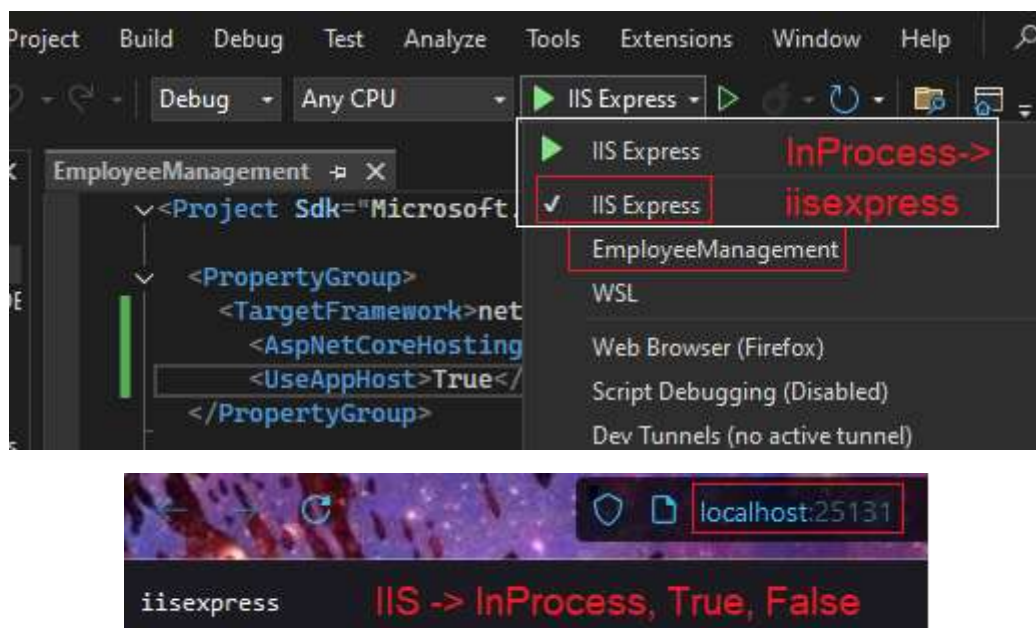
```
C:\Program Files\dotnet\sdk\9.0.200\Microsoft.Common.CurrentVersion.targets(4885,5): error MSB3027: Could not copy "obj\De
bug\net5.0\EmployeeManagement.dll" to "bin\Debug\net5.0\EmployeeManagement.dll". Exceeded retry count of 10. Failed. The f
ile is locked by: ".NET Host (19248)"
C:\Program Files\dotnet\sdk\9.0.200\Microsoft.Common.CurrentVersion.targets(4885,5): error MSB3021: Unable to copy file "o
bj\Debug\net5.0\EmployeeManagement.dll" to "bin\Debug\net5.0\EmployeeManagement.dll". The process cannot access the file '
D:\Exercises\12_class\EmployeeManagement\EmployeeManagement\bin\Debug\net5.0\EmployeeManagement.dll' because it is being u
sed by another process.

The build failed. Fix the build errors and run again.

D:\Exercises\12_class\EmployeeManagement\EmployeeManagement>
```

Ако се върнем във VS, можем да изберем 2 сървъра за стартиране:

- 1) този с името на проекта
- 2) IIS Express – той ще се стартира с по-голям порт като номерация и ще изпише процеса при InProcess:



Следващ урок: тестване на MyKey за приоритети на настройващите файлове.

В Startup.cs създадохме празен конструктор с **ctor** + Tab, създадохме нова променлива config и с **Ctrl + .** за автокорекции създадохме необходимото частно поле private field за декларирането ѝ.

```
Startup.cs x
EmployeeManagement > Startup.cs > Startup > Configure
14 public class Startup
15 {
16     2 references
17     private IConfiguration _config;
18
19     0 references
20     public Startup(IConfiguration config)
21     {
22         _config = config;
23     }
24 }
```

Закоментираме с Ctrl + K командата за извеждане на процеса и вместо него, искаме да изпише стойността на MyKey

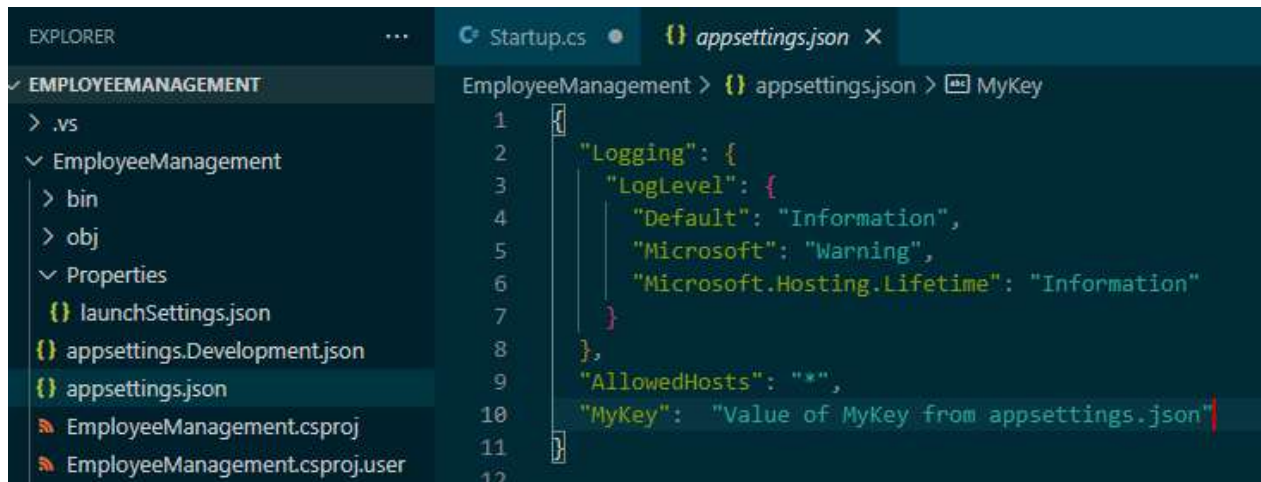
```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        // await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().ProcessName);
        await context.Response.WriteAsync(_config["MyKey"]);
    });
});
```

`await context.Response.WriteAsync(_config["MyKey"]);`

Ще напишем променливата MyKey с текстов стринг в 3-те различни настройващи файла .json и тестваме техните приоритети:

```
3 launchSettings.json
1 appsettings.json
2 appsettings.Development.json
```

Може да се тества дори с букви в 3-те .json файла и в стринговете да пише само А, В, С

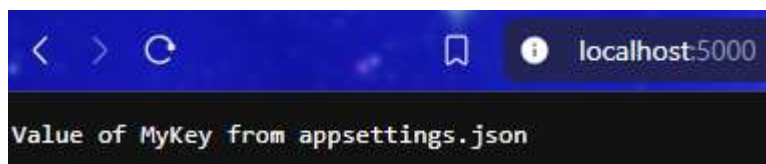


```
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft": "Warning",
6       "Microsoft.Hosting.Lifetime": "Information"
7     }
8   },
9   "AllowedHosts": "*",
10  "MyKey": "Value of MyKey from appsettings.json"
11 }
12
```

В appsettings.json слагаме запетая , на предходния ред и дописваме

```
"MyKey": "Value of MyKey from appsettings.json"
```

Сера, когато стартираме с dotnet run, това ще се изпше в браузъра

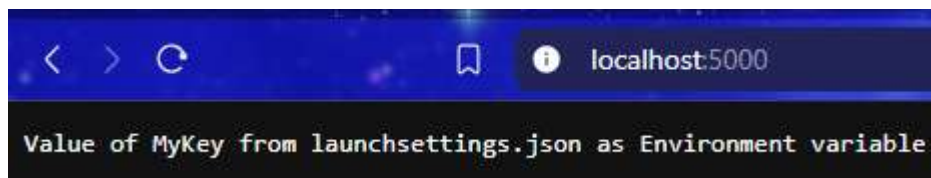


Нека допишем същия код под локалната променлива за средата в launchSettings, като не забравяме запетайката на предния ред и в текста поясним променливата от кой файл се зарежда:

```
"MyKey": "Value of MyKey from launchsettings.json as Environment variable"
```

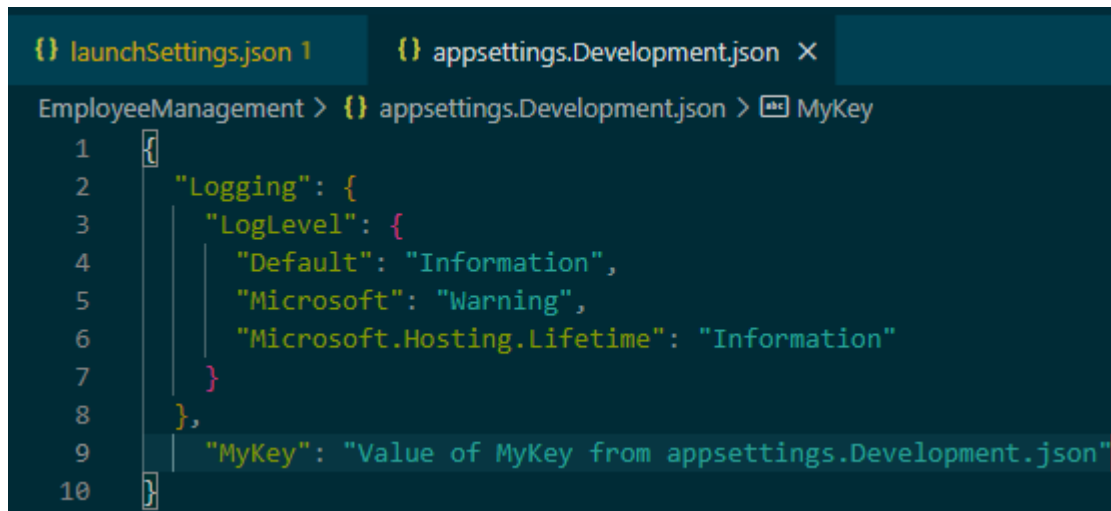
```
launchSettings.json 1 x
EmployeeManagement > Properties > launchSettings.json > profiles > EmployeeManagement > environment
10  "profiles": {
11    "IIS Express": {
12      "commandName": "IISExpress",
13      "launchBrowser": true,
14      "environmentVariables": {
15        "ASPNETCORE_ENVIRONMENT": "Development",
16        "MyKey": "Value of MyKey from launchsettings.json as Environment variable"
17      }
18    },
19    "EmployeeManagement": {
20      "commandName": "Project",
21      "dotnetRunMessages": "true",
22      "launchBrowser": true,
23      "applicationUrl": "http://localhost:5000",
24      "environmentVariables": {
25        "ASPNETCORE_ENVIRONMENT": "Development",
26        "MyKey": "Value of MyKey from launchsettings.json as Environment variable"
27      }
28    }
29  }
30 }
31
```

Резултат:



Т.е. до момента, в Startup направихме променлива в MyKey и съответно я извикахме да се изпише в брауъра, а после я сложихме в системните настройки със съответните текстове от двата файла appsettings и launchsettings и видяхме, че втората е с по-висок приоритет, затова, макар MyKey да присъства и в двата файла, се изпълнява променливата на средата, а не на програмата (environment в launch е по-важно от app).

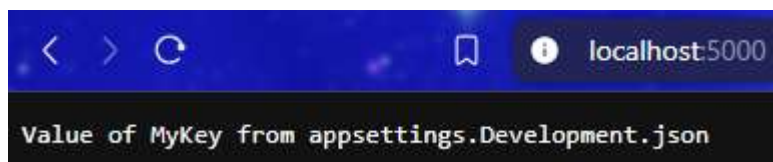
Декларираме същата променлива в appsettings.Development.json с пояснителен текст:



```
EmployeeManagement > {} appsettings.Development.json > MyKey
1  {
2    "Logging": {
3      "LogLevel": {
4        "Default": "Information",
5        "Microsoft": "Warning",
6        "Microsoft.Hosting.Lifetime": "Information"
7      }
8    },
9    "MyKey": "Value of MyKey from appsettings.Development.json"
10 }
```

"MyKey": "Value of MyKey from appsettings.Development.json"

Като презаредим браузъра, нищо не се случва, но ако закоментираме във Visual Studio (във Visual Code коментарите се считат за грешки - там можем да изтрием), ще се види.



Важно е да запомним, че този IConfiguration service предоставен от framework, ще прочете конфигурационните файлове в този ред като последователност:

- 1) Files appsettings.json
 - appsettings.{Environment}.json
- 2) User secrets
- 3) Environment variables – launchsettings.json
- 4) Command-line arguments CLI (max)

Нека проверим CLI, като зададем директорията и `dotnet run MyKey="Value from Command Line"`

Ако забравим да затворим терминала от Visual Studio или Code, ще хвърли грешка:

```
Select Developer Command Prompt for VS 2022
*****
** Visual Studio 2022 Developer Command Prompt v17.10.4
** Copyright (c) 2022 Microsoft Corporation
*****

C:\Program Files\Microsoft Visual Studio\2022\Community>cd /d D:\Exercises\12 class\EmployeeManagement\EmployeeManagement

D:\Exercises\12 class\EmployeeManagement\EmployeeManagement>dotnet run MyKey="Value from Command Line"
Using launch settings from D:\Exercises\12 class\EmployeeManagement\EmployeeManagement\Properties\launchSettings.json...
Building...
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warni
ng NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Pleas
e refer to https://aka.ms/dotnet-core-support for more information about the support policy.
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warni
ng NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Pleas
e refer to https://aka.ms/dotnet-core-support for more information about the support policy.
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warni
ng NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Pleas
e refer to https://aka.ms/dotnet-core-support for more information about the support policy.
crit: Microsoft.AspNetCore.Server.Kestrel[0]
      Unable to start Kestrel.
      System.IO.IOException: Failed to bind to address http://127.0.0.1:5000: address already in use.
        --> Microsoft.AspNetCore.Connections.AddressInUseException: Only one usage of each socket address (protocol/netwo
rk address/port) is normally permitted.
        --> System.Net.Sockets.SocketException (10048): Only one usage of each socket address (protocol/network address/p
ort) is normally permitted.
```

Затова първо Ctrl + C .

```
Developer Command Prompt for VS 2022 - dotnet run MyKey="Value from Command Line"

D:\Exercises\12 class\EmployeeManagement\EmployeeManagement>dotnet run MyKey="Value from Command Line"
Using launch settings from D:\Exercises\12 class\EmployeeManagement\EmployeeManagement\Properties\launchSettings.json...
Building...
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warni
ng NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Pleas
e refer to https://aka.ms/dotnet-core-support for more information about the support policy.
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warni
ng NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Pleas
e refer to https://aka.ms/dotnet-core-support for more information about the support policy.
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warni
ng NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Pleas
e refer to https://aka.ms/dotnet-core-support for more information about the support policy.
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\Exercises\12 class\EmployeeManagement\EmployeeManagement
```



С което доказваме, че с най-най-висок приоритет е CLI!!!

Program.cs файла съдържа Main-метод, който е входната точка на апликацията. Този метод извиква CreateWebHostBuilder, който от своя страна извиква CreateDefaultBuilder – това е методът, който съдържа наборът на разчитане на последователността по подразбиране на всички конфигурационни източници.

The image shows a Visual Studio code editor window with the file 'Program.cs' open. The code is for a class named 'Program'. It has a 'Main' method that calls 'CreateHostBuilder(args).Build().Run()'. There is also a 'CreateHostBuilder' method that is annotated with a lightbulb icon, suggesting a warning or a suggestion. The code is as follows:

```
12 public class Program
13 {
14     0 references
15     public static void Main(string[] args)
16     {
17         CreateHostBuilder(args).Build().Run();
18     }
19     1 reference
20     public static IHostBuilder CreateHostBuilder(string[] args) =>
21     {
22         Host.CreateDefaultBuilder(args)
23         .ConfigureWebHostDefaults(webBuilder =>
24         {
25             webBuilder.UseStartup<Startup>();
26         });
27     }
28 }
```

Понеже .NET core е open source, ние можем да намерим кода на този WebHostClass в github-библиотеката:

Версия 2.2 е максимална към момента:

<https://github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs>

Проверих за по-новата 5.0, но няма

aspnet / MetaPackages Public archive

<> Code Issues Pull requests Projects Wiki Security Insights

release/2.2 MetaPackages / src / Microsoft.AspNetCore / WebHost.cs / <> Jump to

Switch branches/tags

release

Branches Tags

release/1.0

release/1.1

release/2.0

release/2.1

✓ release/2.2

View all branches

(#287) X

1 rights reserved.

Version 2.0. See License.txt in the project root for lic

8 using Microsoft.AspNetCore.HostFiltering;

Приоритетът се определя от open-source метод `CreateDefaultBuilder` извикан във файл `Program.cs`

```
public static IHostBuilder CreateHostBuilder(string[] args) =>
    Host.CreateDefaultBuilder(args)
        .ConfigureWebHostDefaults(webBuilder =>
        {
            webBuilder.UseStartup<Startup>();
        });
```

<https://github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore/WebHost.cs>

С търсачката в страницата търсим **ConfigureAppConfiguration** – методът, в който е описана последователността, която се залага по подразбиране за разчитане на всички конфигурационни

ИЗТОЧНИЦИ:

 github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs

```
.ConfigureAppConfiguration((hostingContext, config) =>
{
    var env = hostingContext.HostingEnvironment;

    config.AddJsonFile("appsettings.json", optional: true, reloadOnChange: true)
        .AddJsonFile($"appsettings.{env.EnvironmentName}.json", optional: true, reloadOnChange: true);

    if (env.IsDevelopment())
    {
        var appAssembly = Assembly.Load(new AssemblyName(env.ApplicationName));
        if (appAssembly != null)
        {
            config.AddUserSecrets(appAssembly, optional: true);
        }
    }

    config.AddEnvironmentVariables();

    if (args != null)
    {
        config.AddCommandLine(args);
    }
}
```

ConfigureAppConfiguration 1/1

Ако искаме, можем да променим тази последователност и дори да добавим наш собствен конфигурационен източник.